数字逻辑设计

实验 1 至 4 学习讲义

从电平、时钟到寄存器、流水灯与数码管

周方璟

目录

1 先认识你正在设计的东西 4
1.1 四个实验各增加了什么能力 . . . 4
1.2 FPGA、开发板和 Verilog 分别是什么 . . . 4
1.3 0 和 1、位宽与二进制 . . . 5
1.4 与、或、非和选择 . . . 5
1.5 时钟、状态和复位 . . . 6
1.6 复位与使能承担不同职责 . . . 6

2 读代码时先辨认电路结构 7
2.1 端口、连线与模块实例 . . . 7
2.2 连续赋值、组合过程与时序过程 . . . 7
2.3 为什么组合逻辑用等号，时序逻辑用非阻塞赋值 . . . 8
2.4 常见符号要结合位置理解 . . . 8
2.5 哪些代码只在仿真里使用 . . . 8

3 实验 1：把当前输入变成当前结果 9
3.1 先按电路的输入输出读它 . . . 9
3.2 逐句理解核心代码 . . . 9
3.3 在板上把数字拨出来，再看 LED 结果 . . . 10
3.4 看仿真波形时沿着一个变化追踪 . . . 10
3.5 这一实验常见的误解 . . . 10

4 实验 2：保存数据，并规定什么时候改写 11
4.1 D 触发器保存一位数据 . . . 11
4.2 寄存器堆：八个地址，每个地址保存八位 . . . 12
4.3 跟着一次写入和两次读取走完全过程 . . . 13
4.4 怎样读实验 2 的波形 . . . 13
4.5 把“保存”与“显示”分开理解 . . . 13

5 实验 3：数时钟，定时移动灯的位置 14
5.1 计数器是“寄存器加一” . . . 14
5.2 从目标时间算上限 . . . 14
5.3 灯的位置如何移动，双灯图案如何生成 . . . 15
5.4 运行中切换是什么意思 . . . 15
5.5 复位时为什么双灯档也只有一盏灯亮 . . . 15
5.6 上板操作、仿真参数和波形判断 . . . 16
5.7 仿真为什么把上限改成 6 和 11 . . . 16

6 实验 4：把按键、计数和显示接成系统 17
6.1 先把顶层分成两条数据通路 . . . 17

6.2 模块清单与各自保存的状态 . . . 17
6.3 按键输入为什么需要同步和消抖 . . . 18
6.3.1 输入同步处理时钟沿附近的不确定性 . . . 18
6.3.2 消抖处理接触过程中的重复变化 . . . 18
6.4 边沿检测：把按住的电平变成一次通知 . . . 19
6.5 通知产生与下游计数不是同一个瞬间 . . . 19
6.6 十进制计数与启停：数字是怎样产生的 . . . 20
6.7 为什么把个位和十位分开保存 . . . 20
6.8 S2 切换的到底是什么 . . . 20
6.9 固定学号的写法为何用了十六进制标记 . . . 20
6.10 数码管：选哪一位，再决定亮哪些段 . . . 21
6.11 八位扫描是怎样循环的 . . . 21
6.12 扫描频率、左右段选和真实操作 . . . 22
6.13 按一次、停一次、切一次显示 . . . 22

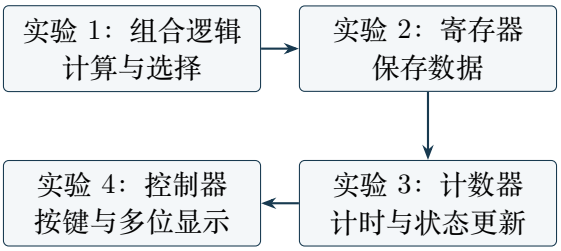
7 把知识用到操作、波形和提问中 23
7.1 从源码到灯亮，中间发生了什么 . . . 23
7.2 XDC 为什么影响实物，却不决定加减规则 . . . 23
7.3 读波形与排查问题时，先缩小到一条因果链 . . . 24
7.3.1 先确认时间和进制 . . . 24
7.3.2 组合逻辑先看输入，时序逻辑先找时钟沿 . . . 24
7.4 代码没有写 CPU 式循环，扫描为何一直重复 . . . 24
7.5 自测题：先自己推演，再看下一页 . . . 25
7.6 试着完整讲一次 S3 按下的过程 . . . 25
7.7 自测参考与一页复习索引 . . . 26

A RTL 代码 27
A.1 mux . . . 27
A.2 dff . . . 28
A.3 reg8file . . . 29
A.4 counter . . . 30
A.5 flowing_water_lights . . . 31
A.6 lab4 . . . 33
A.7 button_debounce . . . 35
A.8 edge_detect . . . 36
A.9 bcd_counter . . . 37
A.10 cycle_counter . . . 38
A.11 led_ctrl_unit . . . 39

1 先认识你正在设计的东西

1.1 四个实验各增加了什么能力

实验 1 接收两个数字，再根据开关选择结果。实验 2 让电路保存数据。实验 3 把保存的数据逐拍加一，用它衡量时间，再控制灯的位置。实验 4 将按键处理、计数和显示连在一起，形成一个能交互的小系统。



实验	要回答的问题	输出取决于什么
1	现在应该输出加法还是减法结果?	当前输入与控制开关
2	之前存了什么，现在允许改写吗?	保存的数据、写入条件和时钟
3	什么时候移动，下一步亮哪里?	计数进度、当前位置和控制开关
4	怎样把数显示出来，把一次按下识别出来?	多个模块各自的状态与连接

1.2 FPGA、开发板和 Verilog 分别是什么

FPGA 是一种可以配置内部逻辑功能与连接方式的芯片。EGO1 是开发板的名称，板上除了 FPGA，还有时钟源、开关、按键、LED 和数码管。你的目标器件是 `xc7a35tcsg324-1`。

Verilog 是硬件描述语言。你用它描述输入输出关系、寄存器以及这些部分如何连接。Vivado 再把描述转成能放进 FPGA 的电路结构。可综合的代码决定硬件；测试代码负责在电脑里制造输入、检查输出。

例如，两个独立的 `always` 块通常代表两部分同时工作的硬件。读源码时从上到下看，方便理解；电路运行时，这些模块会并行工作。实验 4 里计数器继续计数的同时，显示模块也一直在扫描。

1.3 0 和 1、位宽与二进制

数字电路用不同电压范围表示 0 和 1，分别称为低电平和高电平。0 和 1 是逻辑值，高电平对应的实际电压由器件和接口标准决定；不必把所有数字电路的 1 都理解成同一个电压。

一位二进制数据叫一位，英文是 bit。一个信号可以只有一位，例如 `en`；也可以有多位，例如 `input_a[3:0]`。后者有四位，从高位到低位依次是第 3、2、1、0 位。

$$(0101)_2 = 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 5.$$

四位无符号数有 $2^4 = 16$ 种组合，范围是 0 至 15。八位有 256 种组合，范围是 0 至 255。这里的“无符号”表示不为负数单独分配含义。

写法	位数	数值	读法
`1'b1`	1	1	一位、二进制的 1
`4'b0101`	4	5	四位、二进制的 0101
`4'd5`	4	5	四位、十进制的 5
`8'hA5`	8	165	八位、十六进制的 A5

十六进制用 0 至 9 及 A 至 F 表示 16 个数字。一个十六进制数字正好对应四个二进制位，所以 `8'hA5` 也就是 `8'b10100101`。波形中用十六进制显示八位 LED 图案，会比写八个二进制字符紧凑。

数字的含义取决于用途。`8'b00000101` 若作为无符号整数，表示 5；若接八个 LED，则表示第 0 和第 2 个灯亮。灯图案里的每一位直接控制一盏灯，这时没必要把它转换成十进制。

位宽会限制结果。4 位输出只能保留 4 位信息：$15 + 1 = 16 = (10000)_2$，只保留低 4 位就得到 0000。$2 - 5 = -3$ 的低 4 位是 1101；按无符号方式看是 13，按 4 位补码方式看才是 -3。你的实验 1 没有额外的符号或溢出指示灯。

1.4 与、或、非和选择

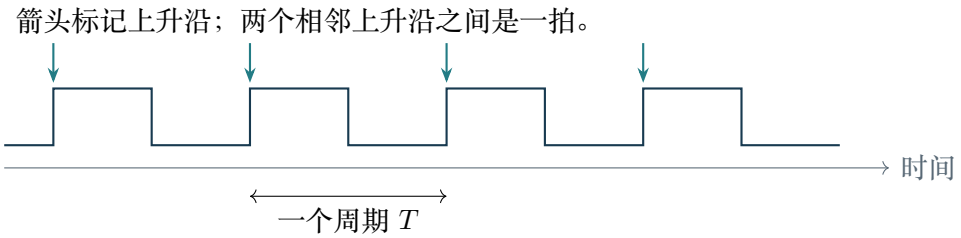
“与”要求两个条件同时成立，例如允许计数且已经到上限；“或”表示至少一个条件成立；“非”把 0、1 反转。Verilog 中的 `&`、`|`、`~` 是按位操作。对于一位信号，可以直接用这三个含义理解。

`sel ? value_a : value_b` 表示：`sel=1` 时选 `value_a`，为 0 时选 `value_b`。它会描述一个选择电路，后面几个实验都会用到。

1.5 时钟、状态和复位

有些电路只根据当前输入给出结果，例如加法器。另一些电路要保存过去的信息，例如已经按了几次、当前亮到第几盏灯。保存的信息称为**状态**。寄存器是保存状态的一类硬件。

时钟 `clk` 周期性地从 0 变成 1，再回到 0。0 变成 1 的瞬间叫上升沿，1 变成 0 的瞬间叫下降沿。时序电路在约定的边沿更新寄存器，你的几个实验统一使用上升沿。



频率 f 表示每秒有多少个周期，周期 $T = 1/f$。100MHz 等于 100×10^6Hz，因此

$$T = \frac{1}{100 \times 10^6}\ \mathrm{s} = 10\ \mathrm{ns}.$$

1 秒等于 1000 毫秒，1 毫秒等于 1000 微秒，1 微秒等于 1000 纳秒。你按住按钮一秒，内部时钟已经走过一亿拍。

时钟沿之间，寄存器的输出保持；它的输入可以由组合电路不断计算。到了下一个时钟沿，寄存器保存输入值。一个常见结构就是“用旧状态计算下一状态，再在时钟沿一起更新”。

1.6 复位与使能承担不同职责

复位把状态放回指定起点。你的 `rst` 和 `clr` 都按高电平有效设计：为 1 时复位。异步复位意味着复位一旦有效，寄存器就响应，不需要等下一个时钟沿。实际硬件仍有传播延迟。

使能决定这一拍是否允许更新。例如 `en=0` 时，D 触发器保持原值；`cnt_end=0` 时，流水灯保持位置。禁用后时钟仍然在运行。

组合电路没有要恢复的历史状态，因此实验 1 不需要复位。时序电路需要从已知状态开始，实验 2 清零存储内容，实验 3 把灯放回 LED0，实验 4 把各计数清零并恢复自动运行。真实寄存器要求输入在时钟沿之前和之后分别保持一小段时间，对应建立时间和保持时间。输入变化若发生在采样沿附近，就可能影响寄存器稳定地采到 0 或 1。

2 读代码时先辨认电路结构

2.1 端口、连线与模块实例

```
module dff (
    input  wire clk,
    input  wire clr,
    input  wire en,
    input  wire d,
    output reg  q
);
```

module dff 声明一个名为 dff 的模块。括号里的内容是它与外界连接的端口，input 表示输入，output 表示输出。这几个信号没有写位宽，所以每个都是一位。模块末尾用 endmodule 结束。

wire 常用于连线或被连续赋值驱动的信号；reg 允许信号在过程块中赋值。单看 reg 这个词，无法断定一定生成寄存器。例如实验 1 的输出也是 reg，但完整的组合赋值最终描述组合电路。是否保存状态，要结合赋值的触发方式和分支是否完整判断。

```
dff u_dff (
    .clk(clk),
    .clr(clr),
    .en(en),
    .d(d),
    .q(q)
);
```

这是在更大的设计里放置一个 dff 模块，u_dff 是这个实例的名字。.clk(clk) 左边是子模块的端口名，右边是外部连接的信号名；两个名字相同只是这里的命名选择。实验 4 里的多个 bcd_counter 实例各自拥有独立状态，连接的使能不同，计数也就不同。

2.2 连续赋值、组合过程与时序过程

```
assign cnt_inc = enable;

always @(*) begin
    q = regfile[rsel];
end

always @(posedge clk or posedge clr) begin
    if (clr) q <= 1'b0;
    else if (en) q <= d;
    else q <= q;
end
```

assign 描述持续成立的连线关系。always @(*) 表示相关输入变化时重新求值，适合完整描述组合逻辑。最后一个块在时钟上升沿或清零上升沿响应，并在清零有效期间保持清零优先。

q <= q 表示保持当前值，不会让数值再增加一次。课程保留完整的“复位、计满、自增、保持”分支，因此读到保持分支时，直接把它理解为本拍不改变状态即可。

2.3 为什么组合逻辑用等号，时序逻辑用非阻塞赋值

组合块里的 = 叫阻塞赋值。同一个块按语句顺序计算，后面的语句能使用前面刚算出的值。组合块应该把所有输出在各条路径上交代清楚，否则可能意外推导出锁存器，即一种会保存状态的硬件。

时序块里的 <= 叫非阻塞赋值。理解本课程代码时，可以按“这一拍先读取旧值，再一起更新目标寄存器”来判断。这个规则对实验 4 的两级寄存器尤其重要。

```
sig_r0 <= signal;
sig_r1 <= sig_r0;
```

假设本拍之前 sig_r0=0，此时输入 signal=1。执行这一拍后，sig_r0 变成 1，sig_r1 拿到的仍是旧的 sig_r0=0。下一拍它才变成 1。如果两级在同一拍都拿到最新输入，就无法保留相邻拍的差别。

2.4 常见符号要结合位置理解

写法	在本项目中的含义
==、>=	比较是否相等、是否大于等于，结果是一位条件
&&	逻辑与，判断两个条件是否同时为真
&、|、~	按位与、或、取反
[7:0]	八位范围；也可从已有信号中取一段位
{led_pos[6:0], led_pos[7]}	拼接：取原第 6 至 0 位放高七位，再把原第 7 位放到最低位
{WIDTH{1'b0}}	代码中的外层拼接与内层重复写法，把 0 重复 WIDTH 次
parameter	配置模块的参数，在展开设计时确定
case	根据输入值选择对应分支，常用于译码

拼接的实际代码见实验 3，注意花括号里左边的项放在结果高位，右边的项放在低位。重复表达式 {WIDTH{1'b0}} 总共生成 WIDTH 个 0。

2.5 哪些代码只在仿真里使用

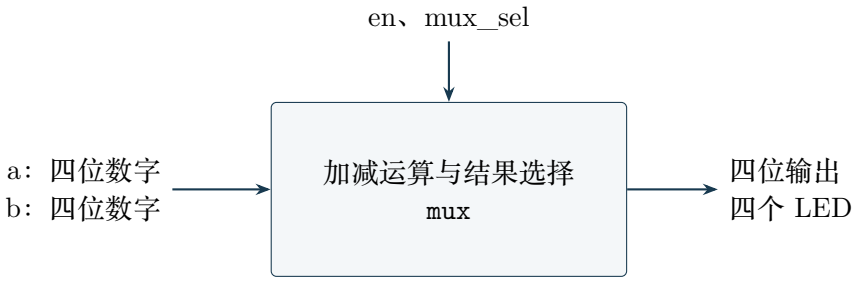
测试文件 testbench 负责产生时钟、复位、开关变化和按键抖动。always ##5 clk = ~clk; 表示每 5 个时间单位翻转时钟，完整周期为 10。配合 `timescale 1ns / 1ps，它就是 10ns 时钟周期。

##20 是仿真延时，$finish 结束仿真，$dumpvars 记录波形；这些不能当成上板 RTL 里的硬件计时方案。RTL 要等 20ms，应使用计数器。课程限制 RTL 不用 initial 初始化，testbench 中的 initial 则是在安排测试过程。

3 实验 1：把当前输入变成当前结果

3.1 先按电路的输入输出读它

实验 1 的顶层是 `mux`，文件是 `lab1/mux.v`。它有两组四位数据输入 `input_a`、`input_b`，一个使能 `en`，一个选择输入 `mux_sel`，以及四位输出 `output_c`。



en	mux_sel	输出	含义
0	任意	1111	未启用时输出规定常量
1	0	a+b 的低 4 位	选择加法结果
1	1	a-b 的低 4 位	选择减法结果

这里“使能无效”的输出由题目与代码定义，恰好是四个 1；使能这个名称本身并不规定禁用时必须输出 0。

3.2 逐句理解核心代码

```
always @(*) begin
    if (en == 1'b0) begin
        output_c = 4'b1111;
    end else if (mux_sel == 1'b0) begin
        output_c = input_a + input_b;
    end else begin
        output_c = input_a - input_b;
    end
end
```

第一行说明它根据输入变化重新计算。第一层 `if` 优先检查使能，禁用时不再看选择信号；启用后才决定采用加法还是减法。三条分支都对 `output_c` 赋值，所以这个块不需要记忆上次输出。

从硬件看，加减与选择逻辑共同实现这一关系。Vivado 可以优化具体实现，不能仅凭源码里写了加号和减号，就断言最终一定保留两个互相独立的运算器。

3.3 在板上把数字拨出来，再看 LED 结果

实验 1 有两类拨码，容易混淆。小拨码 SW_DIP0 控制 `en`，SW_DIP1 控制 `mux_sel`；大拨码 SW3 至 SW0 给 a，SW7 至 SW4 给 b。下表都按逻辑 0、1 描述，实物方向应先核对。

假设要设置 a=5、b=2，就要使大拨码从 SW7 到 SW0 的输入为 `0010 0101`。其中 SW0 是 a 的最低位，SW4 是 b 的最低位。

操作	要检查什么	预期结果
固定 a=5、b=2，将 en 设为 0	禁用分支优先	输出 1111，四个灯都亮
en 改为 1，mux_sel 设为 0	加法分支	输出 0111，即第 0、1、2 位灯亮
保持输入，mux_sel 改为 1	减法分支	输出 0011，即第 0、1 位灯亮
加法模式改为 a=15、b=1	四位结果截断	输出 0000
减法模式改为 a=2、b=5	负结果的位模式	输出 1101，不会出现单独的负号

3.4 看仿真波形时沿着一个变化追踪

先找到 `en`、`mux_sel`、a、b 和 c 这几条信号，确认显示进制。a、b 若用十六进制显示，A 代表十进制 10；c 若显示 F，就对应四位全 1。

以 a=5、b=2 为例：`en=0` 时 c 应为 F；`en` 升为 1 且选择加法后 c 应为 7；随后只改变 `mux_sel`，c 应为 3。这个实验没有时钟，判断输出不需要找上升沿。RTL 仿真通常把组合变化显示在同一仿真时刻，真实电路要经过传播时间。

3.5 这一实验常见的误解

“reg 输出一定有存储器吗？”这里没有。`reg` 是过程赋值所需的声明类型，完整的 `always @(*)` 仍是组合电路。

“if 是不是 CPU 执行完前一句再执行后一句？”综合后的电路持续实现这个选择关系。你改变开关，就改变了选择逻辑的输入。

“为什么 16 显示成 0？”因为只接出了四位结果。若要完整表达 0 至 30 的加法结果，需要至少五位；当前课程接口是四位，因此按低四位观察。

能自己解释这三个输入输出例子，就已抓住实验 1：禁用输出 F；5 加 2 输出 7；5 减 2 输出 3。接下来加上寄存器，电路才能在输入变化后保留旧数据。

4 实验 2：保存数据，并规定什么时候改写

4.1 D 触发器保存一位数据

D 触发器的 d 是数据输入，q 是保存后的输出。当前代码的优先级是：清零优先；没有清零时，若使能有效，在时钟上升沿写入；否则保持。

```
module dff (
    input  wire clk,
    input  wire clr,
    input  wire en,
    input  wire d,
    output reg  q
);
    always @(posedge clk or posedge clr) begin
        if (clr) begin
            q <= 1'b0;
        end else if (en) begin
            q <= d;
        end else begin
            q <= q;
        end
    end
endmodule
```

端口声明说明五个信号各占一位。`posedge clk` 表示时钟上升沿；`posedge clr` 允许清零在两个时钟沿之间也生效。块内部先检查 `clr`，所以它和时钟同时有效时仍然优先清零。

`q <= d` 把本拍采到的 d 保存到 q。假设 d 从 0 变成 1，但变化时刻没有时钟上升沿，q 暂时不变；到下一上升沿，若 en=1，q 才变成 1。若随后 en=0，d 再怎么变化，q 都保留 1，直到允许写入或清零。

事件	clr	en	d	q 的行为
clr 从 0 变 1	1	任意	任意	清零，不等时钟
时钟上升沿	0	1	1	更新为 1
两个上升沿之间 d 改为 0	0	1	0	仍为 1
下一个时钟上升沿	0	1	0	更新为 0
再到一个上升沿	0	0	1	保持 0

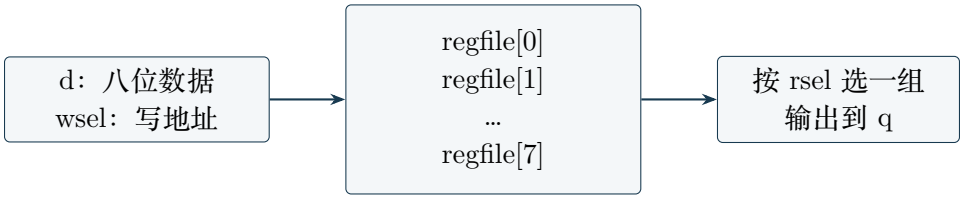
判断时序行为时，先看复位，再看是不是有效时钟沿，最后看使能和输入。

4.2 寄存器堆：八个地址，每个地址保存八位

```
reg [7:0] regfile [7:0];
```

第一个 [7:0] 说明每份数据有八位，第二个 [7:0] 说明有八份数据，编号 0 至 7。因此总容量是 $8 \times 8 = 64$ 位，等于 8 字节。它在本实验中可以理解为八组八位寄存器；综合器最终采用何种资源，还会受读写与复位方式影响。

wsel 和 rsel 都是三位，因为三位恰好能表达 0 至 7 这八个地址。地址是“存在哪里”，数据是“存什么”，两者不需要有相同位宽。



clk、clr、en 控制写入

写入部分位于时序块。清零时把八组数据全部设为 0；允许写入时，执行：

```
regfile[wsel] <= d;
```

例如 wsel=3'd2、d=8'hA5，下一个时钟上升沿把 A5 存到 2 号地址。其他地址没有被改写，会保持原来的数据。课程模板中的保持分支显式写出 regfile[wsel] <= regfile[wsel]。

读出部分则是组合块：

```
always @(*) begin
    q = regfile[rsel];
end
```

只要 rsel 改变，q 就根据新的读地址重新选择输出，不需要等时钟。因此这个设计叫**同步写、组合读**，也常称同步写、异步读。这里“异步读”特指读取不用等时钟，与后文输入同步所解决的问题含义不同。

若读写同一个地址，写入的时钟沿到来前读到旧值；该沿完成写入后，组合读输出随新的寄存器内容变化。讨论这个顺序时要分清沿前与沿后，真实硬件还有输出传播延迟。

4.3 跟着一次写入和两次读取走完全过程

在仿真中依次改变清零、写使能、地址和输入数据，可以观察一次完整的存取过程。

1. 先将 clr 置 1，八个地址清零；读取任意地址，预期都是 00。
2. 撤销 clr，置 en=1，令 wsel=2、d=A5。有效时钟沿到来之前，2 号地址仍为 00；沿后写入 A5。
3. 令 wsel=5、d=3C，等待写入上升沿。此时 5 号地址保存 3C，2 号地址继续保存 A5。
4. 将 en 置 0，再改变 d 为 FF。这只是改变了输入，两个地址已保存的数据不会被覆盖。
5. 令 rsel=2，q 输出 A5；令 rsel=5，q 输出 3C。切读地址无需等待写入时钟沿。
6. 在两个时钟上升沿之间再次把 clr 拉高，读出的内容就变成 00，用来证明清零确实是异步的。

4.4 怎样读实验 2 的波形

先从 clk 找到每个上升沿，再在沿前读 clr、en、wsel 和 d。由这些值判断这一拍会不会写、写到哪里、写入什么。接着看沿后的保存内容与 q。观察组合读时，换一条线索：先找 rsel 何时改变，再看 q 有没有选中对应地址。

仿真里 `x` 通常表示未知值，例如未初始化的寄存器或冲突；`z` 表示高阻状态。它们不是普通的 0 或 1。实验开始的复位就是把本来未必确定的状态带到已知起点。输出未知时先检查复位和连接，再判断功能。

4.5 把“保存”与“显示”分开理解

读取 2 号地址不会把 A5 搬走，也不会清空它。q 只是所选数据的输出。切换 rsel 到 5，相当于换一个观察对象，2 号地址仍保存 A5。这个区分在实验 4 里同样适用：切换显示模式也不会自动清空后台计数。

D 触发器和寄存器堆之间的区别主要是规模和选择机制。一个 D 触发器保存一位，八位寄存器保存八位，寄存器堆还加入地址选择。后面的计数器同样依靠寄存器保存数据，只是写进去的数据通常由旧值加一得到。

自检：en=0、clr=0 时，只改变 d，q 会改变吗？D 触发器不会。寄存器堆若仅改变 d，也不会改变所存数据；但即使 en=0，只要改变 rsel，q 仍可能变，因为读地址选中了另一份已保存的数据。

5 实验 3：数时钟，定时移动灯的位置

5.1 计数器是“寄存器加一”

`counter.v` 保存计时进度，`flowing_water_lights.v` 保存灯的位置并选择输出图案。顶层例化一个 counter，二者共用 100MHz 时钟。

计数器的规则很明确：复位时归零；计满时归零；允许计数时加一；否则保持。当前实验 3 把 `cnt_inc` 固定为 1，表示不暂停计时。

```
assign cnt_max = freq_ctrl ? CNT_MAX_FAST : CNT_MAX_SLOW;
assign cnt_end = cnt_inc & (cnt >= cnt_max);
assign cnt_inc = 1'b1;

always @(posedge clk or posedge rst) begin
    if (rst)          cnt <= {WIDTH{1'b0}};
    else if (cnt_end) cnt <= {WIDTH{1'b0}};
    else if (cnt_inc) cnt <= cnt + 1'b1;
    else              cnt <= cnt;
end
```

`cnt_max` 由速度选择决定，`cnt_end` 说明当前已经允许执行“计满后的更新”。它由组合比较产生，在正常满计数状态持续一拍；下一个时钟沿计数器清零，灯的位置在同一个时钟沿更新。

例如上限为 6 时，状态按 `0,1,2,3,4,5,6,0,...` 循环。0 和 6 都各占一个时钟周期，所以相邻两次计满更新相隔 7 拍。

5.2 从目标时间算上限

若每隔 T 秒更新一次，时钟频率为 f_{clk}，所需拍数 $N = f_{\text{clk}}T$。从 0 计到 $N-1$，上限为

$$\text{CNT_MAX} = f_{\text{clk}}T - 1 = \frac{f_{\text{clk}}}{f_{\text{update}}} - 1.$$

用途	更新间隔	拍数 N	计数上限
实验 3 慢档	0.1s	10,000,000	9,999,999
实验 3 快档	0.002s	200,000	199,999
题目要求 20Hz 换灯	0.05s	5,000,000	4,999,999

这里每计满一次就移动一次灯，因此不再除以 2。如果另一个任务规定“每计满一次翻转一个方波输出”，一次完整方波周期包含两次翻转，那才需要把两个半周期考虑进去。

5.3 灯的位置如何移动，双灯图案如何生成

led_pos 保存一个只有一位为 1 的八位图案。复位后为 00000001，表示 LED0。编号递增时，它依次变成：

$$01 \rightarrow 02 \rightarrow 04 \rightarrow 08 \rightarrow 10 \rightarrow 20 \rightarrow 40 \rightarrow 80 \rightarrow 01.$$

这里每一项是十六进制。80 是 10000000，再走一步时最高位绕回最低位。这叫循环移位。

```
assign next_pos = ctrl_sync[1] ? {led_pos[0], led_pos[7:1]}
                               : {led_pos[6:0], led_pos[7]};
assign next_pair = next_pos | {next_pos[6:0], next_pos[7]};
```

ctrl_sync[1]=0 时，原第 0 位移动到第 1 位，原第 7 位回到第 0 位；为 1 时按相反方向移动。花括号拼接位，| 将两个图案按位合并。

例如 next_pos=00000100，循环左移一位得到 00001000，两者按位或就是 00001100，第 2 和第 3 个灯一起亮。若 next_pos=10000000，相邻位绕回最低位，双灯就是 10000001。

程序单独保存 led_pos，再根据模式生成 led。这样切回单灯时，仍能找到明确的移动位置。当前代码中双灯始终由位置位和其编号递增的相邻位组成；切换运动方向时，两盏灯的整体移动方向改变。

5.4 运行中切换是什么意思

外部开关先经 ctrl_meta、ctrl_sync 两级采样，再用于内部控制。速度和方向变化无需重新复位；单双灯图案在下一个有效换灯沿更新。同步采样带来几个时钟周期以内的处理过程，相对 0.1 秒的慢档很短。

计数判断采用 >= 很有用：若当前 cnt 已经数到 500 万，此时由慢档改快档，新上限只有 199999。>= 能识别已经越过新上限并安排清零；仅用 == 可能错过新上限，直到计数器回绕后才重新遇到它。

5.5 复位时为什么双灯档也只有一盏灯亮

复位分支直接把 led_pos 和 led 都写成 8'h01，优先级高于模式。因此按住 S1 时只有 LED0 亮；松开后等下一次计满，再按当前模式更新图案。这个现象正好能用来说明复位优先级。

5.6 上板操作、仿真参数和波形判断

观察 LD2_0 至 LD2_7。S1 是高电平复位，SW0 切速度，SW1 切方向，SW7 切单双灯。以下 0、1 均指逻辑值，不凭板子摆放方向猜开关上下。

步骤	操作	目的与预期现象
1	SW0=0、SW1=0、SW7=0；按住 S1	设慢速、编号递增、单灯；复位时只亮 LED0
2	松开 S1，观察约 2 秒	每 0.1s 移动一次，0 至 7 再回 0
3	只将 SW1 改为 1，不复位	从当前位置向编号递减方向移动
4	只将 SW7 改为 1，不复位	下次更新出现相邻双灯，继续反向移动
5	将 SW1 改回 0，再将 SW7 改回 0	依次观察恢复正向、恢复单灯
6	将 SW0 改为 1，再改回 0	快档每 2ms 换位；回慢档后又能看清逐灯移动
7	SW7=1 时按住 S1，再松开	按住仍只亮 LED0；释放后恢复双灯运行

快档的 500Hz 是每秒换位 500 次，八个位置完整轮过一遍是 62.5 次每秒。肉眼可能看到多个灯近似同时亮，不能据此认定逻辑上同时输出了多位 1。慢档适合看位置和图案，快档主要观察速度变化。

5.7 仿真为什么把上限改成 6 和 11

真实慢档一次更新需要 1000 万拍，若波形窗口只显示几十拍，就几乎看不到变化。testbench 通过命名参数覆盖，把快档设为 6、慢档设为 11，仍保留相同的控制规则，便于看到完整计数周期。

```
flowing_water_lights #(
    .WIDTH(4),
    .CNT_MAX_FAST(4'd6),
    .CNT_MAX_SLOW(4'd11)
) u_flowing_water_lights ( /* named ports */ );
```

这是参数用法摘录，省略了已在实际测试文件中完整连接的端口。100MHz 仿真时，快档更新间隔为 70ns，慢档 120ns。上板仍使用默认 199999 和 9999999，不能把仿真缩短值直接当成真实时间参数。

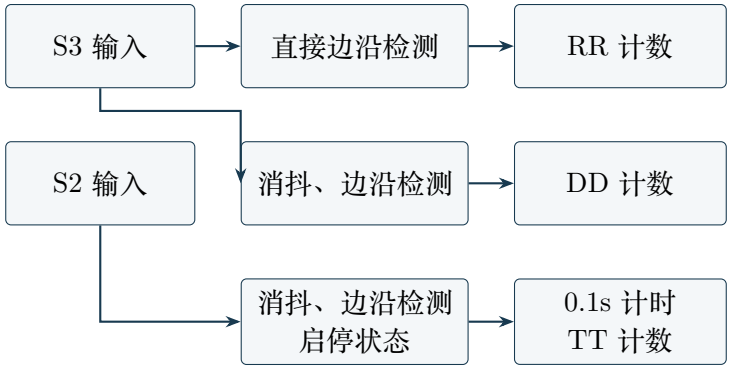
波形先看复位，再看速度切换，最后看模式切换。cnt 设为无符号十进制，检查 0 至 6、0 至 11 都完整出现；led 看十六进制，81 表示 LED7、LED0，03 表示 LED1、LED0。方向与模式变化后都不再按复位，才能证明运行中切换有效。

计数器和灯寄存器共同接原始 clk。cnt_end 接在 if 条件里，意思是“这一拍允许换灯”。若把它放进 posedge 事件里当新时钟，就改变了电路的时钟结构，也违背本课程要求。

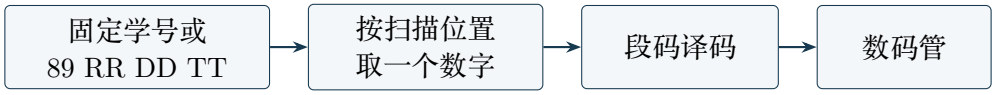
6 实验 4：把按键、计数和显示接成系统

6.1 先把顶层分成两条数据通路

实验 4 顶层文件和模块都叫 `lab4`。一条通路处理按键并生成数字；另一条通路把选中的数字转换成数码管输出。各模块持续并行工作。



RR 与 DD 各按 00 至 99 循环，TT 按 00 至 20 循环。显示数据由模式开关选择：逻辑 1 固定显示 `25310489`，逻辑 0 从左到右显示 `89 RR DD TT`。RR 与 DD 来自同一个 S3 按键。



模式开关只决定展示哪组内容，计数逻辑在后台继续运行。S2 控制 TT 的启停，不控制数码管扫描；暂停自动计数时，所有数码管仍应稳定显示。

6.2 模块清单与各自保存的状态

文件	任务与状态
`lab4.v`	连接子模块；保存模式同步值和 running 启停状态
`button_debounce.v`	同步按键；保存稳定计数与确认后的按键电平
`edge_detect.v`	保存相邻拍采样值，生成上升沿通知
`bcd_counter.v`	分别保存十位和个位
`cycle_counter.v`	保存周期计时进度，支持使能暂停
`led_ctrl_unit.v`	保存扫描位置，组合选择数据并译码

6.3 按键输入为什么需要同步和消抖

6.3.1 输入同步处理时钟沿附近的不确定性

人按下 S3 的时刻与 100MHz 时钟没有固定关系。如果输入恰好在采样沿附近变化，寄存器可能暂时进入无法立即稳定为 0 或 1 的状态，称为亚稳态。两级同步给前一级更多稳定时间，降低这种不确定性继续传播的概率；它不能把风险绝对消除。

```
button_meta <= button;
button_sync <= button_meta;
```

第二级保存的是前一级上一拍的值。后面的消抖逻辑使用 button_sync，而非直接使用外部 button。几拍的延迟只有几十纳秒，对人的按键操作几乎感觉不到。

6.3.2 消抖处理接触过程中的重复变化

机械按键在按下和松开时可能短暂反复接通、断开。原始采样可能是 0,1,0,1、1,0,1,1,...。如果每次 0 变 1 都计一次，人按一次就可能加好几次。

本设计保存确认过的电平 button_clean。若新采样和它一致，计数器清零；若不同，就开始统计这种不同连续保持了多少拍。达到 20ms 才把新电平接受为 button_clean。如果中途又回到原电平，累计进度取消，重新判断。

```
if (rst) cnt <= 21'd0;
else if (button_sync == button_clean) cnt <= 21'd0;
else if (cnt == DEBOUNCE_MAX) cnt <= 21'd0;
else cnt <= cnt + 21'd1;
```

另一个时序块在 button_sync != button_clean 且 cnt == DEBOUNCE_MAX 时更新确认电平。这两个块在同一时钟沿读取旧值，所以归零计数和接受新状态可以同时完成。

100MHz 下 20ms 需要 $2,000,000$ 拍，默认上限为 1,999,999。这个窗口按同步后的采样统计，外部按键到确认输出还包含同步与时钟采样的延迟；不要把外部按下到确认的全过程机械地当成恰好 20ms。

按下与松开都需要经过同样的稳定判断，这叫双向消抖。如果还没确认松开就立刻再次按下，电路可能认为一直按住，无法形成第二次有效按下。正常分开的按压应该给按下和松开都留出稳定时间。

同步和消抖解决不同问题。两级寄存器只延迟几十纳秒，无法过滤毫秒级机械抖动；20ms 消抖也不能代替前面的异步输入同步。实验 4 把两步分别实现，读代码时也要分别判断。

6.4 边沿检测：把按住的电平变成一次通知

即使经过消抖，按键按住一秒时，button_clean 仍会高电平持续约一秒。若写成“只要电平为 1 就加一”，就会在很多个时钟沿重复计数。因此需要识别由 0 到 1 的那个变化。

```
`timescale 1ns / 1ps
module edge_detect (
    input wire clk,
    input wire rst,
    input wire signal,
    output wire pos_edge
);
    reg sig_r0;
    reg sig_r1;
    // 两级寄存
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            sig_r0 <= 1'b0;
            sig_r1 <= 1'b0;
        end else begin
            sig_r0 <= signal;
            sig_r1 <= sig_r0;
        end
    end
    assign pos_edge = sig_r0 & ~sig_r1;
endmodule
```

sig_r0 保存较新的采样，sig_r1 保存上一拍的 sig_r0。sig_r0 & ~sig_r1 要求“新采样是 1，旧采样是 0”，恰好对应上升沿。

以下假定复位后两级为 0，输入在第一拍之前稳定变为 1。表格给出每个时钟沿更新完成之后的值。

时刻	输入	sig_r0	sig_r1	pos_edge
第一拍之前	1	0	0	0
第一拍之后	1	1	0	1，产生一拍宽通知
第二拍之后	1	1	1	0，长按不再重复通知
稳定按住期间	1	1	1	0
松开采样之后	0	0	1	0，下降沿不算按下
再下一拍	0	0	0	0，准备识别下次按下

6.5 通知产生与下游计数不是同一个瞬间

第一拍更新后 pos_edge 才变成 1。同样使用上升沿的下游 BCD 计数器，在第一拍读到的仍是此前的 0；到第二拍，它读到沿前为 1 的通知，才完成加一。同时边沿检测器把通知降回 0。理解这个顺序，就能解释为什么计数波形比采样寄存器再晚一个沿。

本课程要求使用这段两级寄存器检测结构。对于 RR 支路，它直接对原始 S3 采样、不做毫秒级消抖，因此会记录被采到的多次上升变化。严格的跨时钟域工程设计还会限制第一级亚稳态输出的使用方式；这里的两级相邻采样比较遵循课程模板，不能把它概括为所有异步输入场景的完整防护方案。

6.6 十进制计数与启停：数字是怎样产生的

6.7 为什么把个位和十位分开保存

二进制整数 20 写成八位是 `00010100`。如果每四位直接当一个待显示的十进制数字，就会显示 1 和 4。为了方便逐位显示，这个设计使用 BCD：每个十进制数字占四位，十位 2、个位 0 分别存放，总图案为 `0010 0000`。

`ones` 和 `tens` 各有四位，但按十进制规则限制在 0 至 9。`bcd_counter` 的优先级是：复位清零；到指定上限且收到计数使能时清零；否则收到使能时加一；没有使能时保持。

```
assign cnt_end = cnt_inc & (tens == MAX_TENS)
                         & (ones == MAX_ONES);
```

未到上限时，若个位为 9，就把个位清零、十位加一，例如 09 到 10；若个位小于 9，就只把个位加一。RR、DD 使用默认上限 99，TT 实例把 `MAX_TENS` 设为 2、`MAX_ONES` 设为 0，因此 20 之后回到 00。

到 20 那一拍不会立刻又清零。计数状态 20 会一直保持到下一次计数使能，因此在自动运行中它也保持 0.1 秒。00 至 20 共有 21 个显示值，一轮需要 2.1 秒。

6.8 S2 切换的到底是什么

S2 先消抖，再生成一次 `s2_press` 通知；顶层收到通知时执行：

```
if (rst) running <= 1'b1;
else if (s2_press) running <= ~running;
else running <= running;
```

`~running` 把 0、1 互换，所以一次按下切换一次运行状态。复位直接设为 1，故松开复位后自动开始，无需另外按启动。

`running` 连接到 0.1 秒周期计数器的 enable。停下时，周期计数器保留当前进度，TT 保持；继续后从剩余进度接着走。例如停在约 40ms 处，恢复后约再走 60ms 就会更新一次，之后每 100ms 更新一次。按键识别本身有延迟，因此这里用“约”来解释时间关系。

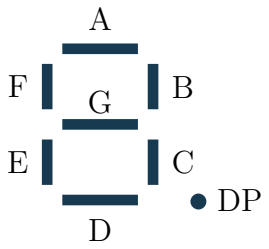
若一次暂停操作恰好与计满更新发生在同一个沿，所有寄存器仍按沿前的状态判断，这一沿可能完成最后一次更新。观察实物时，应在暂停按键被确认后再检查是否持续停止。

6.9 固定学号的写法为何用了十六进制标记

`32'h25310489` 把八个待显示数字依次装进八个四位字段。扫描器每次取四位，直接译成一位数码管。它没有先把十进制整数 25310489 转换成普通二进制整数再拆位。顶层拼接 `89 RR DD TT` 时使用同样的逐位格式。

6.10 数码管：选哪一位，再决定亮哪些段

一位数码管用 A 至 G 七个发光段组成数字，另有小数点 DP。显示 1 点亮 B 和 C，显示 8 点亮 A 至 G，显示 0 点亮除 G 以外的六段。



段选顺序：A B C D E F G DP
高电平点亮，DP 默认关闭。

代码把段选写成八位 `segment`，顺序是 `{A,B,C,D,E,F,G,DP}`。显示 1 的段码 `8'h60` 即 `01100000`，B 和 C 两位为 1；显示 8 为 `8'hFE` 即 `11111110`，七段亮、小数点灭。

6.11 八位扫描是怎样循环的

`scan_index` 有三位，能表示 0 至 7。每计满 1ms 增加一次，到 7 再加一时三位结果回到 0。位置 0 选择最右边一位，位置 7 选择最左边一位。

扫描编号	位选 led_en	从 display 取出的字段	固定学号下的数字
0	01	[3:0]	9，最右位
1	02	[7:4]	8
2	04	[11:8]	4
3	08	[15:12]	0
4	10	[19:16]	1
5	20	[23:20]	3
6	40	[27:24]	5
7	80	[31:28]	2，最左位

实际扫描从右往左走，人眼看到的是左至右的 25310489。`led_en` 决定哪一位亮，段码决定这一位的字形；只给段码、不选位，或选位与数字字段错配，都不能正确显示。

6.12 扫描频率、左右段选和真实操作

每位 1ms，一轮八位就是 8ms。因此切位频率是 1000Hz，每位每秒刷新 125 次，固定内容下每位占一轮的八分之一时间。人眼的时间积分使快速轮询形成稳定显示。125Hz 是本设计的整轮刷新率，不能把 1kHz 写成每一位的刷新率。

EGO1 的左右四位有独立段选。当前代码中 `led_cx` 控制右四位，`led_cx1` 控制左四位：

```
assign led_cx  = scan_index[2] ? 8'h00 : segment;
assign led_cx1 = scan_index[2] ? segment : 8'h00;
```

位置 0 至 3 时 `scan_index[2]=0`，把段码送到右组，左组置 0；位置 4 至 7 时反过来。位选与段选都按高电平有效设计。

6.13 按一次、停一次、切一次显示

S1 复位、S2 启停、S3 计数。SW0 逻辑 1 为固定模式，逻辑 0 为指定模式；课程要求向上固定、向下指定，首次上板仍需核对物理方向。

步骤	具体操作	目的与预期现象
1	按 S1 再松开，SW0 设为 1	八位显示 25310489，检查顺序与各段
2	SW0 改为 0，再按 S1 松开	显示 89 00 00 TT，TT 从 00 开始每 0.1s 增加
3	按 S2、松开，等约 1 秒	TT 停止；扫描和其他显示继续工作
4	再按 S2 并松开	TT 从保留的计时进度继续，20 后回 00
5	按 S3 一次并充分松开，再按一次	DD 依次到 01、02；RR 可能相同，也可能更多
6	长按 S3 约 1 秒，再松开	DD 只增加 1；稳定长按不应逐拍累加
7	不复位，来回切 SW0	固定与指定显示随时切换，后台计数保留
8	最后再按 S1 并松开	RR、DD 清零，TT 重新从 00 自动运行

RR 是否多计取决于实际抖动有没有被采到，一次按键恰好只计一次完全可能。不要为了证明消抖，声称实物每次必然多计。短到不足稳定窗口的按压可能被 DD 忽略，这是过滤规则的一部分。

仿真将 `SCAN_MAX=9`、`DEBOUNCE_MAX=9`、`TICK_MAX=99`。在 10ns 时钟下，它们分别对应每位 100ns、稳定判定 10 拍、自动计时 100 拍。波形中的纳秒级消抖是缩短参数后的演示，真实上板依旧使用 20ms 窗口。

7 把知识用到操作、波形和提问中

7.1 从源码到灯亮，中间发生了什么

1. 写设计文件，规定电路的输入输出、寄存器和更新规则。顶层模块负责把各部分接起来。
2. 写 testbench，在电脑里提供输入，做行为仿真。它适合检查加减选择、时钟沿写入、计满更新、抖动过滤等规则。
3. 配置约束。`pin.xdc` 把顶层端口对应到芯片封装引脚，并指定接口电平标准；时序设计还要给工具说明时钟周期。比如 100MHz 对应 10ns。
4. 检查代码并执行综合，把 RTL 转换成逻辑资源和寄存器的连接。RTL 图帮助理解结构，综合后电路图展示优化后的结构，二者不一定长得一样。
5. 执行实现，把资源放置到芯片具体位置并完成布线，然后检查时序是否满足目标时钟要求。
6. 生成 bitstream 配置文件，通过 Hardware Manager 下载到开发板，按照操作表检查真实现象。

仿真正确说明测试覆盖到的逻辑行为符合预期。真实板上还涉及引脚、电平、时钟约束、机械输入与实现时序，因此仿真通过后仍需综合、实现和上板。

7.2 XDC 为什么影响实物，却不决定加减规则

实验 1 里的 `input_a[0]` 是逻辑端口，约束把它接到 EGO1 大拨码 SW0 对应的 R1 引脚。若逻辑正确、引脚却接错，板上操作就对不上。反过来，换一个引脚并不会把加法规则变成减法，规则仍来自 RTL。

仿真波形中使用的是逻辑端口名，不会因为没有连到实际拨码就不能给它赋值。仿真测试和真实板上输入是两种不同的输入来源。

7.3 读波形与排查问题时，先缩小到一条因果链

7.3.1 先确认时间和进制

看清时间单位是 ns 还是 ms。仿真用了缩短参数时，先用实际覆盖值计算周期。二进制 `00010000` 与十六进制 `10` 是同一组位，若当成十进制 10 去理解，灯的位置就会判断错误。

7.3.2 组合逻辑先看输入，时序逻辑先找时钟沿

实验 1 从输入变化追到输出。实验 2 先找到写入沿，再查看沿前的使能和数据。实验 3 先找到 cnt 到上限的一拍，再看下一更新沿的 cnt 和 led。实验 4 则把路径缩小到 `button -> clean -> press -> count`，逐级核对。

多级寄存器不要期待所有信号在同一时刻改变。前级刚产生的值通常要到下一时钟沿才会被下游时序逻辑接收。查看波形时给变化点适当放大，分清沿前值、沿后值以及组合传播。

看到的现象	优先检查什么
实验 1 四个灯一直亮	是否仍在 en=0；是否误拨了大拨码而非使能小拨码
实验 2q 不随 d 改变	当前是否有上升沿、en 是否为 1；寄存器堆是否读取写入地址
实验 3 一直停在 LED0	是否按住复位；时钟是否连对；是否误用巨大仿真上限
实验 3 快档看起来多个灯亮	先回慢档观察；快档的视觉现象不直接等于逻辑图案
实验 4 单个数字不对或缺段	段码位序、有效电平、左右段选与引脚约束
实验 4 数字对，但位置颠倒	位选与 display 切片的对应关系
实验 4 一按增加很多次	当前看的是 RR 还是 DD；消抖参数是否仍是仿真值
实验 4 暂停后数字仍闪或熄灭	是否误停了扫描；当前设计只暂停 TT 计时
波形出现 X	先确认复位、端口连接与是否有多个驱动

7.4 代码没有写 CPU 式循环，扫描为何一直重复

时钟不断提供上升沿。周期计数器在每拍更新，计满时扫描编号加一，三位编号溢出后回 0。各模块从配置后就持续工作，因此无需写软件式 `while` 循环。硬件计数器的状态转移本身就形成重复行为。

7.5 自测题：先自己推演，再看下一页

1. 四位无符号数最大是多少？`8'h81` 接八个 LED，哪些灯亮？
2. 实验 1 中 en=0、a=5、b=2，改变选择开关会改变输出吗？如果 en=1、a=15、b=1 并选加法呢？
3. D 触发器 clr=0、en=1，d 在两个上升沿之间由 0 变 1，q 什么时候更新？en=0 时呢？
4. 寄存器堆的 2 号地址已有 A5，5 号地址已有 3C。en=0 时，把 rsel 从 2 改成 5，q 会变吗？为什么？
5. 100MHz 下每 0.1 秒更新一次，为什么上限是 9,999,999？若上限设为 6，每次更新相隔几拍？
6. 实验 3 慢档运行到 cnt=5,000,000，直接切到快档，`>=` 为什么比 `==` 合适？
7. `led_pos=8'h80` 时，按编号递增方向的下一个位置是什么？若下一位置是 80，对应双灯图案是什么？
8. 边沿检测器原来 r0=r1=0，输入已稳定为 1。第一拍之后 r0、r1 各是多少？第二拍之后呢？
9. 消抖后按键高电平持续一秒，为何仍需要边沿检测？两级同步能直接替代 20ms 消抖吗？
10. 每位显示 1ms，八位一轮是多少 ms、整轮刷新率多少 Hz？扫描编号 0 选中最左还是最右？
11. `32'h25310489` 在这里保存什么格式的数据？把普通整数 20 的二进制 `00010100` 直接拆成两个四位数字，会显示什么？
12. TT 从 00 至 20 每 0.1 秒变化一次，一轮多久？按 S2 暂停时，数码管扫描是否也停止？

7.6 试着完整讲一次 S3 按下的过程

从手指按下开始，沿着“机械触点变化、输入采样、稳定判断、边沿通知、BCD 加一、显示数据、扫描、段码”逐项解释。RR 支路绕过哪一步？哪几步需要等待时钟？屏幕上看到新数字还会受当前扫描位置怎样影响？

7.7 自测参考与一页复习索引

1. 最大 15。81 是 `10000001`，第 7、0 位灯亮。
2. 禁用时始终 1111，选择开关不影响结果。15+1 的低四位为 0000。
3. 下一次时钟上升沿且 en=1 时写入；en=0 时保持原值。clr 有效会优先异步清零。
4. 会从 A5 变为 3C，因为读出是组合选择。en 只控制写入。
5. 0 到 9,999,999 有 1000 万个计数状态，每拍 10ns，共 0.1s。0 到 6 有 7 个状态，更新间隔 7 拍。
6. 旧计数已经超过 199,999，`>=` 能及时识别超出；`==` 可能错过新的上限。
7. 80 循环左移后为 01；位置 80 对应的双灯是 81。
8. 第一拍后 r0=1、r1=0，通知为 1；第二拍后均为 1，通知为 0。下游计数器在第二拍接收到沿前有效的通知。
9. 高电平持续很多拍，电平作使能会重复计数；边沿检测仅给一次通知。两级同步的时间尺度很短，不能代替毫秒级消抖。
10. 一轮 8ms，125Hz。编号 0 选择最右位。
11. 每四位装一个待显示的数字。`00010100` 拆开是 1 与 4，直接译码会显示 14；BCD 的 20 应写成 `00100000`。
12. 共 21 个值，一轮 2.1s。S2 暂停 TT 计时，扫描继续；RR 和 DD 也不受 running 控制。

术语	在本实验中的具体意义
组合逻辑	当前输入决定输出，例如加减选择、段码译码
时序逻辑	寄存器保存状态，并按时钟与复位规则更新
寄存器	保存一组位，例如 cnt、led_pos、ones
使能	允许本拍更新，通常写在 if 条件里
参数	展开设计时确定的配置；testbench 可覆盖为小数值
同步	对来自时钟域外的输入采样，降低亚稳态传播风险
消抖	稳定持续达到窗口才接受新电平
边沿检测	比较相邻采样，将上升变化变成单拍通知
BCD	每个十进制数字单独占四位
位选 / 段选	选择显示位置 / 决定该位置的字形

A RTL 代码

A.1 mux

```
`timescale 1ns/1ps

module mux (
    input  wire       en,
    input  wire       mux_sel,
    input  wire [3:0] input_a,
    input  wire [3:0] input_b,
    output reg  [3:0] output_c
);

    // 根据使能和选择信号确定输出
    always @(*) begin
        if (en == 1'b0) begin
            output_c = 4'b1111;
        end else if (mux_sel == 1'b0) begin
            output_c = input_a + input_b;
        end else begin
            output_c = input_a - input_b;
        end
    end

endmodule
```

A.2 dff

```
module dff (
    input  wire clk,
    input  wire clr,
    input  wire en,
    input  wire d,
    output reg  q
);
    always @(posedge clk or posedge clr) begin
        if (clr) begin
            q <= 1'b0;
        end else if (en) begin
            q <= d;
        end else begin
            q <= q;
        end
    end
endmodule
```

A.3 reg8file

```
module reg8file (
    input  wire       clk,
    input  wire       clr,
    input  wire       en,
    input  wire [7:0] d,
    input  wire [2:0] wsel,
    input  wire [2:0] rsel,
    output reg  [7:0] q
);
    reg [7:0] regfile [7:0];

    always @(posedge clk or posedge clr) begin
        if (clr) begin
            regfile[0] <= 8'h00;
            regfile[1] <= 8'h00;
            regfile[2] <= 8'h00;
            regfile[3] <= 8'h00;
            regfile[4] <= 8'h00;
            regfile[5] <= 8'h00;
            regfile[6] <= 8'h00;
            regfile[7] <= 8'h00;
        end else if (en) begin
            regfile[wsel] <= d;
        end else begin
            regfile[wsel] <= regfile[wsel];
        end
    end

    always @(*) begin
        q = regfile[rsel];
    end
endmodule
```

A.4 counter

```
`timescale 1ns / 1ps

module counter #(
    parameter WIDTH = 24,
    parameter [WIDTH-1:0] CNT_MAX_FAST = 199_999,
    parameter [WIDTH-1:0] CNT_MAX_SLOW = 9_999_999
)(
    input  wire             clk,
    input  wire             rst,
    input  wire             freq_ctrl,
    output reg  [WIDTH-1:0] cnt,
    output wire             cnt_end
);
    wire [WIDTH-1:0] cnt_max;
    wire cnt_inc;

    // 计数条件
    assign cnt_max = freq_ctrl ? CNT_MAX_FAST : CNT_MAX_SLOW;
    assign cnt_end = cnt_inc & (cnt >= cnt_max);
    assign cnt_inc = 1'b1;

    always @(posedge clk or posedge rst) begin
        if (rst)          cnt <= {WIDTH{1'b0}};
        else if (cnt_end) cnt <= {WIDTH{1'b0}};
        else if (cnt_inc) cnt <= cnt + 1'b1;
        else              cnt <= cnt;
    end
endmodule
```

A.5 flowing_water_lights

```
`timescale 1ns / 1ps
module flowing_water_lights #(
    parameter WIDTH = 24,
    parameter [WIDTH-1:0] CNT_MAX_FAST = 199_999,
    parameter [WIDTH-1:0] CNT_MAX_SLOW = 9_999_999
)(
    input  wire       clk,
    input  wire       rst,
    input  wire       freq_ctrl,
    input  wire       dir_ctrl,
    input  wire       mode_sel,
    output reg  [7:0] led
);
    // 拨码同步
    (* ASYNC_REG = "TRUE" *) reg [2:0] ctrl_meta;
    (* ASYNC_REG = "TRUE" *) reg [2:0] ctrl_sync;
    wire [WIDTH-1:0] cnt;
    reg [7:0] led_pos;
    wire cnt_end;
    wire [7:0] next_pos;
    wire [7:0] next_pair;

    // 计数器
    counter #(
        .WIDTH(WIDTH),
        .CNT_MAX_FAST(CNT_MAX_FAST),
        .CNT_MAX_SLOW(CNT_MAX_SLOW)
    ) u_counter (
        .clk(clk),
        .rst(rst),
        .freq_ctrl(ctrl_sync[0]),
        .cnt(cnt),
        .cnt_end(cnt_end)
    );

    // 循环移位
    assign next_pos = ctrl_sync[1] ? {led_pos[0], led_pos[7:1]}
                                   : {led_pos[6:0], led_pos[7]};
    assign next_pair = next_pos | {next_pos[6:0], next_pos[7]};

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            ctrl_meta <= 3'b000;
            ctrl_sync <= 3'b000;
        end else begin
            ctrl_meta <= {mode_sel, dir_ctrl, freq_ctrl};
            ctrl_sync <= ctrl_meta;
        end
    end

    // 换灯控制
    // 位置与输出同步更新
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            led_pos <= 8'h01;
            led <= 8'h01;
        end else if (cnt_end) begin
```

```
58              led_pos <= next_pos;
59              if (ctrl_sync[2]) begin
60                  led <= next_pair;
61              end else begin
62                  led <= next_pos;
63              end
64          end else begin
65              led_pos <= led_pos;
66              led <= led;
67          end
68      end
69  endmodule
```

A.6 lab4

```
`timescale 1ns / 1ps
module lab4 #(
    parameter [31:0] STUDENT_ID = 32'h25310489,
    parameter [16:0] SCAN_MAX = 17'd99_999,
    parameter [20:0] DEBOUNCE_MAX = 21'd1_999_999,
    parameter [23:0] TICK_MAX = 24'd9_999_999
)(
    input wire clk,
    input wire rst,
    input wire button_s2,
    input wire button_s3,
    input wire mode_sel,
    output wire [7:0] led_en,
    output wire [7:0] led_cx,
    output wire [7:0] led_cx1
);
    reg mode_meta;
    reg mode_sync;
    reg running;
    wire s2_clean;
    wire s3_clean;
    wire s2_press;
    wire s3_raw_press;
    wire s3_clean_press;
    wire [3:0] raw_tens, raw_ones;
    wire [3:0] clean_tens, clean_ones;
    wire [3:0] auto_tens, auto_ones;
    wire [23:0] tick_cnt;
    wire tick_end;
    wire [31:0] display;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            mode_meta <= 1'b0;
            mode_sync <= 1'b0;
        end else begin
            mode_meta <= mode_sel;
            mode_sync <= mode_meta;
        end
    end
    button_debounce #(.DEBOUNCE_MAX(DEBOUNCE_MAX)) u_button_debounce_s2 (
        .clk(clk), .rst(rst), .button(button_s2), .button_clean(s2_clean)
    );
    button_debounce #(.DEBOUNCE_MAX(DEBOUNCE_MAX)) u_button_debounce_s3 (
        .clk(clk), .rst(rst), .button(button_s3), .button_clean(s3_clean)
    );
    edge_detect u_edge_detect_s2 (
        .clk(clk), .rst(rst), .signal(s2_clean), .pos_edge(s2_press)
    );
    edge_detect u_edge_detect_raw (
        .clk(clk), .rst(rst), .signal(button_s3), .pos_edge(s3_raw_press)
    );
    edge_detect u_edge_detect_clean (
        .clk(clk), .rst(rst), .signal(s3_clean), .pos_edge(s3_clean_press)
    );
    always @(posedge clk or posedge rst) begin
        if (rst) running <= 1'b1;
        else if (s2_press) running <= ~running;
```

```
58          else running <= running;
59      end
60      bcd_counter u_bcd_counter_raw (
61          .clk(clk), .rst(rst), .cnt_inc(s3_raw_press), .tens(raw_tens), .ones(raw_ones)
62      );
63      bcd_counter u_bcd_counter_clean (
64          .clk(clk), .rst(rst), .cnt_inc(s3_clean_press), .tens(clean_tens), .ones(clean_ones)
65      );
66      cycle_counter #(.WIDTH(24), .CNT_MAX(TICK_MAX)) u_cycle_counter_tick (
67          .clk(clk), .rst(rst), .enable(running), .cnt(tick_cnt), .cnt_end(tick_end)
68      );
69      bcd_counter #(.MAX_TENS(4'd2), .MAX_ONES(4'd0)) u_bcd_counter_auto (
70          .clk(clk), .rst(rst), .cnt_inc(tick_end), .tens(auto_tens), .ones(auto_ones)
71      );
72      assign display = mode_sync ? STUDENT_ID :
73          {STUDENT_ID[7:0], raw_tens, raw_ones, clean_tens, clean_ones, auto_tens, auto_ones};
74      led_ctrl_unit #(.SCAN_MAX(SCAN_MAX)) u_led_ctrl_unit (
75          .rst(rst), .clk(clk), .display(display), .led_en(led_en),
76          .led_cx(led_cx), .led_cx1(led_cx1)
77      );
78  endmodule
```

A.7 button_debounce

```
`timescale 1ns / 1ps
module button_debounce #(
    parameter [20:0] DEBOUNCE_MAX = 21'd1_999_999
)(
    input wire clk,
    input wire rst,
    input wire button,
    output reg button_clean
);
    reg button_meta;
    reg button_sync;
    reg [20:0] cnt;
    // 输入同步
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            button_meta <= 1'b0;
            button_sync <= 1'b0;
        end else begin
            button_meta <= button;
            button_sync <= button_meta;
        end
    end
    // 稳定20ms
    always @(posedge clk or posedge rst) begin
        if (rst) cnt <= 21'd0;
        else if (button_sync == button_clean) cnt <= 21'd0;
        else if (cnt == DEBOUNCE_MAX) cnt <= 21'd0;
        else cnt <= cnt + 21'd1;
    end
    always @(posedge clk or posedge rst) begin
        if (rst) button_clean <= 1'b0;
        else if ((button_sync != button_clean) && (cnt == DEBOUNCE_MAX))
            button_clean <= button_sync;
        else button_clean <= button_clean;
    end
endmodule
```

A.8 edge_detect

```
`timescale 1ns / 1ps
module edge_detect (
    input wire clk,
    input wire rst,
    input wire signal,
    output wire pos_edge
);
    reg sig_r0;
    reg sig_r1;
    // 两级寄存
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            sig_r0 <= 1'b0;
            sig_r1 <= 1'b0;
        end else begin
            sig_r0 <= signal;
            sig_r1 <= sig_r0;
        end
    end
    assign pos_edge = sig_r0 & ~sig_r1;
endmodule
```

A.9 bcd_counter

```
`timescale 1ns / 1ps
module bcd_counter #(
    parameter [3:0] MAX_TENS = 4'd9,
    parameter [3:0] MAX_ONES = 4'd9
)(
    input wire clk,
    input wire rst,
    input wire cnt_inc,
    output reg [3:0] tens,
    output reg [3:0] ones
);
    wire cnt_end;
    assign cnt_end = cnt_inc & (tens == MAX_TENS) & (ones == MAX_ONES);
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            tens <= 4'd0;
            ones <= 4'd0;
        end else if (cnt_end) begin
            tens <= 4'd0;
            ones <= 4'd0;
        end else if (cnt_inc) begin
            if (ones == 4'd9) begin
                tens <= tens + 4'd1;
                ones <= 4'd0;
            end else begin
                tens <= tens;
                ones <= ones + 4'd1;
            end
        end else begin
            tens <= tens;
            ones <= ones;
        end
    end
endmodule
```

A.10 cycle_counter

```
`timescale 1ns / 1ps
module cycle_counter #(
    parameter WIDTH = 24,
    parameter [WIDTH-1:0] CNT_MAX = 24'd9_999_999
)(
    input wire clk,
    input wire rst,
    input wire enable,
    output reg [WIDTH-1:0] cnt,
    output wire cnt_end
);
    wire cnt_inc;
    assign cnt_inc = enable;
    assign cnt_end = cnt_inc & (cnt == CNT_MAX);
    always @(posedge clk or posedge rst) begin
        if (rst)          cnt <= {WIDTH{1'b0}};
        else if (cnt_end) cnt <= {WIDTH{1'b0}};
        else if (cnt_inc) cnt <= cnt + 1'b1;
        else              cnt <= cnt;
    end
endmodule
```

A.11 led_ctrl_unit

```
`timescale 1ns / 1ps
module led_ctrl_unit #(
    parameter [16:0] SCAN_MAX = 17'd99_999
)(
    input wire rst,
    input wire clk,
    input wire [31:0] display,
    output reg [7:0] led_en,
    output wire [7:0] led_cx,
    output wire [7:0] led_cx1
);
    wire [16:0] cnt;
    wire cnt_end;
    reg [2:0] scan_index;
    reg [3:0] data;
    reg [7:0] segment;
    cycle_counter #(.WIDTH(17), .CNT_MAX(SCAN_MAX)) u_cycle_counter (
        .clk(clk), .rst(rst), .enable(1'b1), .cnt(cnt), .cnt_end(cnt_end)
    );
    always @(posedge clk or posedge rst) begin
        if (rst) scan_index <= 3'd0;
        else if (cnt_end) scan_index <= scan_index + 3'd1;
        else scan_index <= scan_index;
    end
    always @(*) begin
        case (scan_index)
            3'd0: begin led_en = 8'h01; data = display[3:0]; end
            3'd1: begin led_en = 8'h02; data = display[7:4]; end
            3'd2: begin led_en = 8'h04; data = display[11:8]; end
            3'd3: begin led_en = 8'h08; data = display[15:12]; end
            3'd4: begin led_en = 8'h10; data = display[19:16]; end
            3'd5: begin led_en = 8'h20; data = display[23:20]; end
            3'd6: begin led_en = 8'h40; data = display[27:24]; end
            3'd7: begin led_en = 8'h80; data = display[31:28]; end
            default: begin led_en = 8'h00; data = 4'd0; end
        endcase
    end
    // 顺序为ABCDEFG和DP
    always @(*) begin
        case (data)
            4'h0: segment = 8'hfc;
            4'h1: segment = 8'h60;
            4'h2: segment = 8'hda;
            4'h3: segment = 8'hf2;
            4'h4: segment = 8'h66;
            4'h5: segment = 8'hb6;
            4'h6: segment = 8'hbe;
            4'h7: segment = 8'he0;
            4'h8: segment = 8'hfe;
            4'h9: segment = 8'hf6;
            4'ha: segment = 8'hee;
            4'hb: segment = 8'h3e;
            4'hc: segment = 8'h9c;
            4'hd: segment = 8'h7a;
            4'he: segment = 8'h9e;
            4'hf: segment = 8'h8e;
            default: segment = 8'h00;
```

```
58          endcase
59      end
60      // 左右独立段选
61      assign led_cx = scan_index[2] ? 8'h00 : segment;
62      assign led_cx1 = scan_index[2] ? segment : 8'h00;
63  endmodule
```